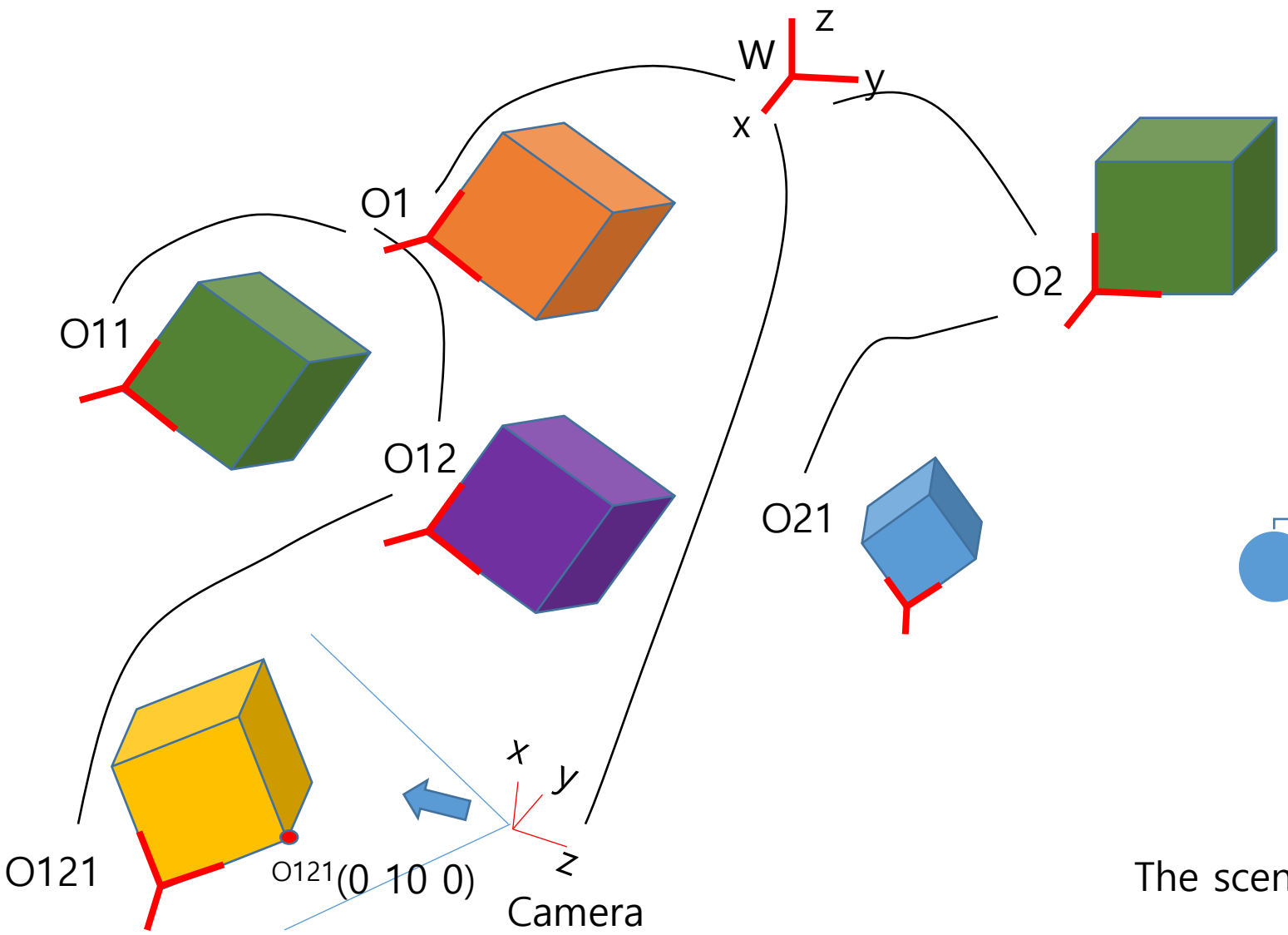
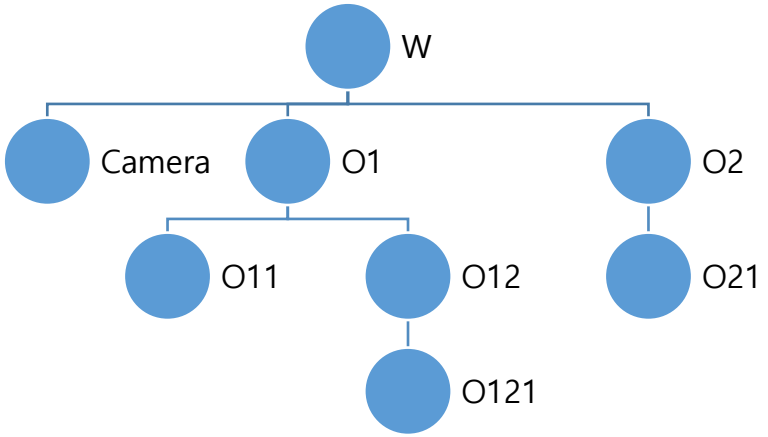
Homework 2: Intro. to Virtual and Augmented Reality

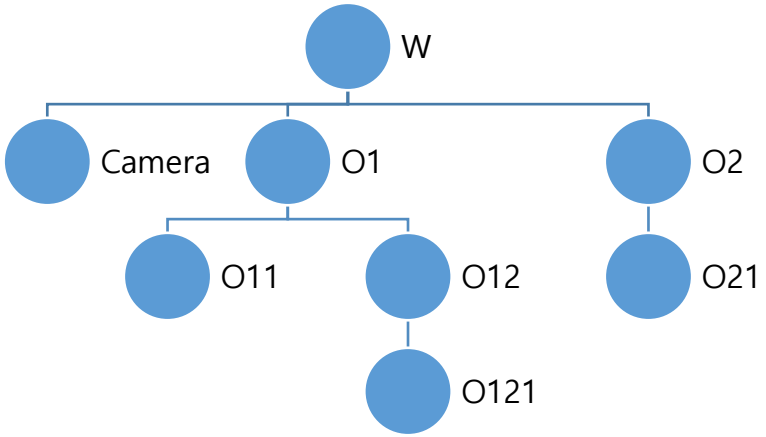
Korea University, Spring 2026

1. In Unity, create a simple scene resembling the situation shown below …





The scene tree would look something like this …



Use arbitrary values for translation and 90 degrees for rotation for simplicity …

E.g. below is just example values …

Parent	Child	Translation (x, y, z)	Euler ZYX	
W	Camera	0, 0, 0	90, 90, 0	Viewing direction –z direction of camera local (Camera is at world)
W	O1	20, 30, 40	90, 0, 90	
W	O2	100, 200, 0	0, 90, 0	
O1	O11	5, 200, 300	0, 0, 90	
O1	O12	30, 40, 0	90, 90, 90	
O12	O121	10, 20, 30	90, 0, 45	
O2	O21	40, 50, 0	90, 45, 0	

Once the scene is completed in Unity, write out the 4x4 transformation matrices between the coordinate systems (e.g. like ${}^{O1}T_{O11}$, …)

Check the attribute window of the object and compare the values with regards to their position and orientation … do they make any sense? Note that Unity by default uses Euler ZYX.
Do values for the orientation/position attributes match your transformation matrix information ?

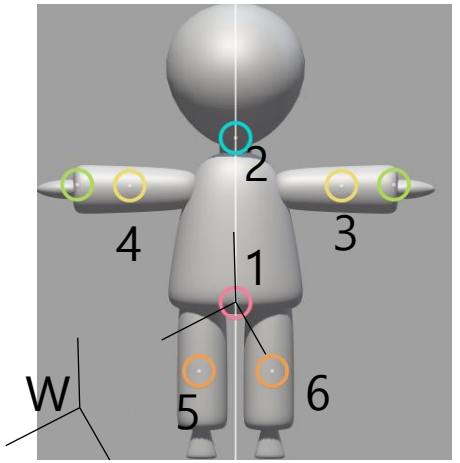
Also note that you can instruct Unity to display the Transformation matrix by writing this code (I got it from AI) … which you can attach to the child object … (see next page) … you can try this (but not requirement)

Take one vertex of O121: ${}^{O121}(0\ 10\ 0)$
How do we convert this coordinate to the one that is with respect to Camera?
${}^{O121}(0\ 10\ 0)$ ➔ ${}^{Camera}(x\ y\ z) = ?$

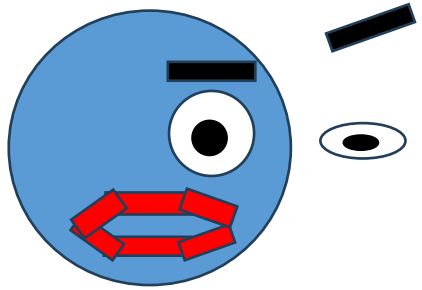
Explain how the depth first traversal of the scene tree will result in rendering only a subset of objects.

```
using UnityEngine;
#if UNITY_EDITOR
using UnityEditor;
#endif

public class MatrixDisplay : MonoBehaviour { }

#if UNITY_EDITOR
[CustomEditor(typeof(MatrixDisplay))]
public class MatrixDisplayEditor : Editor {
    public override void OnInspectorGUI() {
        DrawDefaultInspector();
        MatrixDisplay script = (MatrixDisplay)target;

        // Calculate matrix relative to parent
        Matrix4x4 localToParentMatrix = Matrix4x4.identity;
        if (script.transform.parent != null) {
            localToParentMatrix = script.transform.parent.worldToLocalMatrix * script.transform.localToWorldMatrix;
        } else {
            localToParentMatrix = script.transform.localToWorldMatrix;
        }

        // Display in Inspector
        EditorGUILayout.LabelField("Local to Parent Matrix", EditorStyles.boldLabel);
        GUILayout.TextArea(localToParentMatrix.ToString("F3"));
    }
}
#endif
```

2. Simple character modeling (and importing)



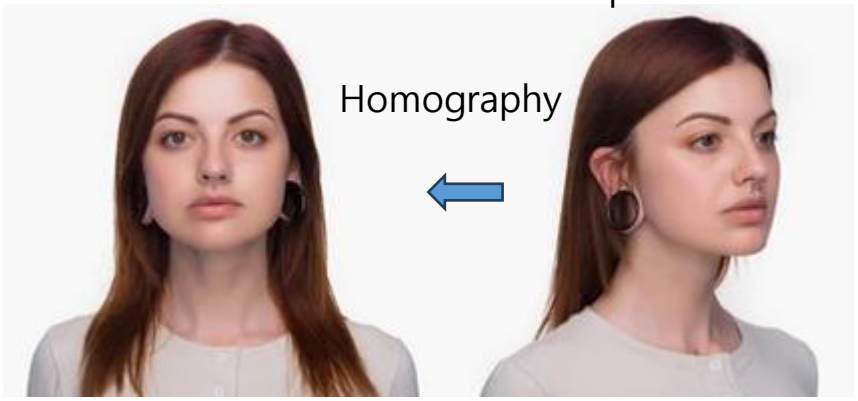
- Design a simple human-like character using a tool like Blender
  - Number of joint may be less than 10 as shown
  - Add eyes, eye brows and mouth
    - Eye: E.g. sphere whose center, eccentricity and size can be changed for blink and expression control (by a script)
    - Eye brow: E.g. a rectangular strip whose position/size can be changed for expression control
    - Lip: E.g. some modifiable shape to express emotion of joy, sadness, anger, etc.



- Assign the local coordinate systems at the joints using the D-H convention
  - Build the D-H parameter table for joints 1 ~ 6
  - What are T's
    - You can assume ${}^{W}T_{1}$ is known or set up so that coordinate systems 1 and W coincide
- Import into Unity and write scripts for …
  - Some simple animations (walk, bow, wave hand, …)
  - Some simple facial expression and animations of the face (e.g. speaking, laughing, frowning, …)

3. Update project scene with two human characters

Take photo like this



Use a tool …

- Import two nice rigged characters (not the one from No. 2)
  - If you have to buy assets, the course can support financially.
- Decorate scene to create the mood for two people (A and B) blind dating …
- Make some simple behaviors like (with animations, facial expression, sound, …):
  - Bowing for A, and another bowing for B
  - Saying Hello/Good bye
  - A: Telling a joke and B: Reacting .. Laughing …
  - A: Telling a bad joke and B: Reacting … Frowning …
  - …
- Homography
  - Take a photo from an arbitrary angle (but cover most of the face) and generate a photo as seen from the front using homography
    - Implement using something like Matlab or Python with OpenCV …
  - Use some tool (e.g. https://www.boundingboxsoftware.com/materialize/) to convert the result to a texture to be applied to the human figures above.